

PipeANN 混合过滤框架正式集成实施计划

GitHub Copilot

2026 年 4 月 23 日

1 文档目标

本文档给出 PipeANN hybrid 过滤框架正式集成的可实施设计，用于将现有实验目录 `hybrid_rebuild_context` 中的只读 hybrid 能力迁入正式源码树，并在正式接线与验证完成后删除整个 `hybrid_rebuild_context` 目录。本文档只保留确定性设计；所有第一阶段和第二阶段工作都必须以本文档中给出的文件落点、接口签名、磁盘文件格式、运行时状态机和验收条件为准。

本文档的约束如下：

1. 第一阶段只覆盖只读索引上的 `intersect` 和 `subset` 查询；`range` 查询继续走现有 `graph-only` 路径；
2. 第一阶段不修改现有 `SSDIndexMetadata` 磁盘头格式；
3. 第一阶段 build 完成后不再依赖原始 base label 的 `.spmat` 文件参与运行时加载；
4. 第一阶段的阈值标定不是“build 代码内部自动发生”的隐式行为，而是 build 完成后的强制后处理步骤；
5. 第二阶段的动态标签更新、live 状态写回和后台重定位必须以前置接口全部落地为起点，不能在缺依赖的情况下并行实现。

2 实施范围

本次实施按两个阶段推进。

第一阶段：正式迁移与只读集成。

1. 将 hybrid 搜索框架从实验目录迁入正式代码目录，即迁入 `include/`、`src/` 和 `tests/`；
2. 在正式查询接口中新增统一 hybrid 查询入口，使 `prefilter` 与 `graph-only` 两条路径由同一套运行时逻辑调度；
3. 将路径切换机制统一为候选规模单阈值 τ_m ，并在第一次索引构建完成后通过独立的后处理标定工具生成初始阈值；
4. 将 τ_m 与标定信息持久化到独立的 hybrid companion metadata 文件 `<index\_prefix>\_hybrid.meta`，并将 base label sidecar 固化为 `<index\_prefix>\_labels.densebit`；
5. 完成正式 binary、正式测试与实验目录替换后，删除 `hybrid_rebuild_context` 目录。

第二阶段：更新路径与后台重定位。

1. 在动态索引具备标签变更接口、DenseBitset live 状态维护能力和统一的 N_{live} 状态源之后，再接入更新路径；
2. 将阈值重定位的唯一触发条件定义为向量数量变化超过 10%，并要求重定位在后台单线程执行，且每条 benchmark 查询开始前都要检测前台负载。

统一 hybrid 路由在第一阶段只覆盖标签过滤 selector_type 为 intersect 与 subset 的查询，不覆盖 range。range 过滤继续沿用现有 graph-only 过滤搜索路径，不接入本计划的 hybrid 路由。

3 总体实施目标

第一阶段集成完成后，PipeANN 中基于标签过滤的查询应具备如下统一执行流程：

1. 查询到达正式查询入口；
2. 运行时根据当前标签过滤条件计算候选规模 $m(F) = |S_F|$ ；
3. 将 $m(F)$ 与候选规模阈值 τ_m 进行比较；
4. 若 $m(F) \leq \tau_m$ ，则进入 prefilter 路径；
5. 若 $m(F) > \tau_m$ ，则进入 graph-only 路径；
6. 两条路径共用同一套索引前缀、标签 sidecar、hybrid metadata、查询统计接口和测试入口。

这里的“标签过滤”仅指 intersect 与 subset 两类 selector。range 不参与本计划中的 hybrid 路由与阈值决策。

对应的路由规则定义为

$$m(F) = |S_F|,$$

$$m(F) = 0 \Rightarrow \text{prefilter-fast-return}, \quad 0 < m(F) \leq \tau_m \Rightarrow \text{prefilter}, \quad m(F) > \tau_m \Rightarrow \text{graph-only}.$$

其中 prefilter-fast-return 表示候选集合为空时直接返回 0 个结果，不进入 PQ shortlist，也不进入 graph 搜索。

4 正式代码迁移方案

4.1 迁移原则

迁移时遵循以下原则：

1. hybrid 相关 C++ 逻辑迁入正式源码树，不再长期保留在实验目录中独立编译；
2. hybrid_rebuild_context 只作为迁移阶段的临时对照源使用，不作为长期保留目录；
3. 统一查询入口放入正式查询接口，而不是继续由 Python 脚本决定选择哪一个独立二进制；
4. 第一阶段不修改现有 SSDIndexMetadata 磁盘头格式，hybrid 阈值与标定信息通过独立 companion metadata 文件管理；
5. 第一阶段所有 hybrid 运行时资产必须只由 index_prefix 派生得到，运行时不得依赖原始 label_file.spmat 的路径；

6. 第一阶段的 DenseBitset sidecar 采用固定命名 `<index\_prefix>\_labels.densebit`，而不是 `<label\_file>.densebit`；
7. 第一阶段的 metadata 采用固定命名 `<index\_prefix>\_hybrid.meta`，并通过原子写入 `.tmp+rename` 更新；
8. 第二阶段能力不阻塞第一阶段落地与实验目录删除；当正式 binary、正式测试和阈值标定流程稳定后，即可整体删除 `hybrid_rebuild_context` 目录及其构建、脚本、缓存、绘图和辅助文件，不保留实验目录。

4.2 阶段划分

为降低与现有代码结构的耦合，本计划按以下阶段实施：

1. **第一阶段（当前执行范围）**：M1、M2、M3、M4、M5、M8。目标是把实验目录里的只读 hybrid 能力迁入正式源码树，形成可调用、可测试、可持久化阈值的正式实现；
2. **第二阶段（后续扩展）**：M6、M7。目标是在动态索引补齐标签变更、live 状态维护和前台负载计数接口后，再接入后台重定位与更新路径。

4.3 模块落点

正式模块按下表拆分并迁移：

模块	正式落点	职责
M1 DenseBitset 运行时	<code>include/filter/densebit_index.h</code> 、 <code>src/filter/densebit_index.cpp</code>	定义 DenseBitset 文件头、sidecar 加载、候选计数、候选物化和查询期 scratch 复用。
M2 Prefilter 执行器	<code>include/filter/hybrid_route.h</code> 、 <code>src/search/hybrid_prefilter.cpp</code>	定义 prefilter PQ 打分、shortlist 维护、SSD 精排和空候选 fast-return。
M3 Hybrid 路由与查询入口	<code>include/ssd_index.h</code> 、 <code>src/search/hybrid_search.cpp</code>	定义统一 <code>hybrid_search(...)</code> 入口、强制路径覆盖、fallback 规则和统计输出。
M4 Metadata 模块	<code>include/filter/hybrid_metadata.h</code> 、 <code>src/filter/hybrid_metadata.cpp</code>	定义 <code><index_prefix>_hybrid.meta</code> 的二进制格式、读写、验证和原子更新。
M5 Build 与标定接线	<code>src/utls/index_build_utils.cpp</code> 、 <code>tests/build_disk_index.cpp</code> 、 <code>tests/build_pipnn_index.cpp</code> 、 <code>src/pipnn.cpp</code> 、 <code>tests/calibrate_hybrid_threshold.cpp</code>	将 base label sidecar 生成并入正式建索引流程，并补充强制执行的后处理标定工具。

M8 正式测试驱动 与清理	tests/search_disk_index_ hybrid.cpp、 tests/CMakeLists.txt、 src/CMakeLists.txt	补充正式 hybrid 驱动、强制路径 测试、构建接线和实验目录删除。
------------------	--	--

1. 在 src/CMakeLists.txt 中将源码 glob 从当前的 *.cppsearch/*.cppupdate/*.cpputils/*.cpp 扩展为包含 filter/*.cpp;
2. 在 include/ssd_index.h 中声明统一 hybrid_search(...) 接口,并新增 load_hybrid_runtime(...) 与 hybrid_enabled();
3. 在 tests/ 中新增正式驱动 tests/search_disk_index_hybrid.cpp 和后处理工具 tests/calibrate_hybrid_threshold.cpp;
4. 在正式源码树中增加 hybrid companion metadata 的读写模块,并在 SSDIndex::load() 中接线自动加载;
5. 在正式建索引流程中增加 <index_prefix>_labels.densebit 的固定命名生成逻辑;
6. 当第一阶段正式 binary、正式测试和阈值标定流程均已稳定后,删除整个 hybrid_rebuild_context 目录,并移除所有对该目录的构建、脚本和文档引用。

4.4 正式产物命名与生命周期

第一阶段 hybrid 运行时只允许使用以下派生产物:

```
<index_prefix>_disk.index
<index_prefix>_labels.densebit
<index_prefix>_hybrid.meta
<index_prefix>_labels.densebit.tmp
<index_prefix>_hybrid.meta.tmp
```

这些文件遵循以下规则:

1. <index_prefix>_labels.densebit 由正式 build 过程根据输入的 label_file.spmat 生成, build 完成后运行时不再依赖原始 .spmat 路径;
2. <index_prefix>_hybrid.meta 只由正式标定工具写入, build 阶段不写默认伪值;
3. sidecar 和 metadata 的写入都必须先写 .tmp 文件, 再 fsync, 最后 rename 覆盖正式文件;
4. SSDIndex::load(index_prefix) 只尝试从上述固定命名文件加载 hybrid runtime;
5. 若 _labels.densebit 或 _hybrid.meta 任何一个缺失、损坏或与磁盘索引不匹配, 则 hybrid runtime 标记为不可用, 查询回退到现有 graph-only 路径;
6. 删除 hybrid_rebuild_context 的前置条件之一是: 正式 build、正式 load、正式 query 全部只依赖这三类正式产物。

4.5 构建与加载接线

为保证第一阶段无需反查原始 label 路径，正式代码必须采用以下接口改动：

```
bool build_disk_index(...,
    const char* tag_file,
    AbstractNeighbor<T>* nbr_handler,
    AbstractLabel* label,
    const char* label_source_file = nullptr);

void create_disk_layout(...,
    const std::string& output_file,
    AbstractLabel* label,
    const std::string& label_source_file = "");

int SSDIndex<T, TagT>::load_hybrid_runtime(const char* index_prefix);
bool SSDIndex<T, TagT>::hybrid_enabled() const;
```

其中：

1. label_source_file 在 label_type == spmat 时必须传入，否则 build 直接失败；
2. build_disk_index(...) 和 PiPNN build 路径都必须把 label_source_file 透传到 sidecar builder；
3. SSDIndex::load() 在核心索引加载完成后调用 load_hybrid_runtime(index_prefix)；
4. search_disk_index_filtered.cpp 保留为 graph-only 基线驱动，tests/search_disk_index_hybrid.cpp 作为正式 hybrid 驱动；
5. tests/calibrate_hybrid_threshold.cpp 负责读取正式索引与查询集，执行阈值标定并写回 _hybrid.meta。

4.6 实验目录删除时机

hybrid_rebuild_context 的删除时机放在迁移收尾阶段，而不是放在模块开发中途。删除前必须同时满足以下条件：

1. 第一阶段的 DenseBitset、prefilter、统一 hybrid 查询入口、阈值标定与正式测试驱动都已迁入正式源码树；
2. 主工程可以独立完成构建、smoke、回归、sidecar 生成和阈值初始化，不再调用 hybrid_rebuild_context 中的 C++ binary、脚本或缓存；
3. 正式测试驱动已经覆盖迁移后的核心路径；
4. 文档中第一阶段模块记录已经补齐正式落地实现，能够独立说明正式实现方式。

第二阶段的后台重定位与更新路径集成不是删除实验目录的前置条件。一旦上述第一阶段条件满足，就直接整体删除 hybrid_rebuild_context 目录，不保留实验副本。

4.7 文档更新规则

本文档需要作为后续正式实现的持续更新接口使用。自本计划开始执行后，每完成一个模块，都必须同步更新本文档中的对应模块记录，至少补齐以下内容：

1. 模块状态，从“未开始”更新为“进行中”或“已完成”；
2. 实际修改的头文件、源码文件、测试文件和构建文件路径；
3. 模块的具体实现方式，包括关键数据结构、控制流与接口设计；
4. 至少一段核心代码片段，用于说明该模块的实际落地方式；
5. 模块验证结果，包括编译、测试、smoke 或回归命令及其结论。

为避免文档退化为简单进度表，后续更新时必须优先补“具体实现方式”和“核心代码片段”，而不仅仅写“已完成”或“已支持”。

5 统一 hybrid 查询入口

统一 hybrid 查询入口作为正式查询接口的一部分实现，而不是脚本层路径分发。第一阶段采用以下对外接口：

```
enum class HybridFilterKind : uint8_t {
kUnsupported = 0,
kIntersect = 1,
kSubset = 2
};

enum class HybridRouteDecision : uint8_t {
kAutoGraphFallback = 0,
kPrefilter = 1,
kGraphOnly = 2,
kPrefilterFastReturn = 3
};

enum class HybridRouteOverride : uint8_t {
kAuto = 0,
kForcePrefilter = 1,
kForceGraphOnly = 2
};

struct HybridQueryStats {
uint64_t candidate_count = 0;
uint64_t threshold = 0;
uint64_t threshold_version = 0;
HybridRouteDecision decision = HybridRouteDecision::kAutoGraphFallback;
```

```

uint64_t route_overhead_us = 0;
};

size_t SSDIndex<T, TagT>::hybrid_search(const T* query,
uint64_t k_search,
uint32_t mem_L,
uint64_t l_search,
TagT* res_tags,
float* res_dists,
uint64_t beam_width,
HybridFilterKind filter_kind,
const void* filter_data,
QueryStats* stats = nullptr,
HybridQueryStats* hybrid_stats = nullptr,
HybridRouteOverride route_override =
HybridRouteOverride::kAuto);

```

该入口完成以下工作：

1. 接收查询向量、标签过滤条件、 k 、 L 、`beamwidth` 等正式参数；
2. 绑定一次性加载的 hybrid runtime，其中包含 DenseBitset sidecar 句柄和 hybrid companion metadata；
3. 基于当前标签过滤条件计算候选规模 $m(F)$ ；
4. 将 $m(F)$ 与当前阈值 τ_m 比较，并记录本次选路结果；
5. 当选择 prefilter 路径时，若已有 bitset 中间结果，则直接复用；
6. 当选择 graph-only 路径时，直接复用现有 `pipe_search(..., selector, filter_data, ...)`；
7. 向统一统计接口输出路径选择、候选规模、阈值版本和查询耗时。

第一阶段统一入口采用以下固定规则：

1. 若 `filter_kind` 为 `kUnsupported`，则直接回退到现有 graph-only 路径；
2. 若 hybrid runtime 不可用，则直接回退到现有 graph-only 路径；
3. 若 `route_override == kForcePrefilter`，则跳过阈值比较，直接进入 prefilter 路径；
4. 若 `route_override == kForceGraphOnly`，则跳过阈值比较，直接进入 graph-only 路径；
5. 若 $m(F) = 0$ ，则直接返回空结果，并将 `decision` 记录为 `kPrefilterFastReturn`；
6. 若 $0 < m(F) \leq \tau_m$ ，则进入 prefilter；
7. 若 $m(F) > \tau_m$ ，则进入 graph-only。

正式驱动 `tests/search_disk_index_hybrid.cpp` 必须支持 `force_route=auto|prefilter|graph` 三种模式，以便对同一批查询分别执行自动路由、强制 prefilter 和强制 graph-only 回归。

6 候选规模计算机制

6.1 查询过滤条件编码

第一阶段复用当前 `SpmatLabel::write()` 的序列化格式, `filter_data` 必须满足以下二进制布局:

```
[count: uint32_t][label_0: uint32_t]...[label_{count-1}: uint32_t]
```

进入 hybrid 路由前, 运行时必须对查询标签做标准化:

1. 将标签复制到本地缓冲区;
2. 按升序排序;
3. 去重;
4. 标准化后再做 bitset AND/OR 和候选物化;
5. 候选 ID 列表始终按 point id 升序输出。

空查询标签的行为固定为:

1. **subset**: $m(F) = N$, 因为空集合是任何目标标签集合的子集;
2. **intersect**: $m(F) = 0$, 因为空标签集合与任何目标标签集合都不构成交集匹配。

6.2 单标签查询

对于单标签查询, 第一阶段直接在 `DenseBitset` sidecar 上读取对应标签行并执行 `popcount`, 得到候选规模。第一阶段不引入单标签专用选择性表, 也不引入单标签额外元数据。该规则同时适用于 `intersect` 与 `subset`, 因为单标签交集和单标签子集在集合意义上等价。

6.3 subset 查询

对于 `subset` 查询, 即要求结果向量至少包含查询中的全部标签, 运行时在 `DenseBitset` sidecar 上执行按位与, 再对结果做 `popcount`, 得到不重复候选数:

$$m(F) = \left| \bigcap_{\ell \in F} S_\ell \right|.$$

若最终选中 `prefilter` 路径, 则复用同一份 bitset 结果继续物化候选 ID; 若最终选中 `graph-only` 路径, 则只保留计数结果参与路由, 不额外物化 ID 列表。

若标准化后的任一标签 ID 超过 `densebit\_header.nlabels-1`, 则 `subset` 直接返回 $m(F) = 0$, 因为不可能存在包含该标签的候选。

6.4 intersect 查询

对于 `intersect` 查询, 运行时在 `DenseBitset` sidecar 上执行按位或, 再对结果做 `popcount`, 得到去重后的候选规模:

$$m(F) = \left| \bigcup_{\ell \in F} S_\ell \right|.$$

这里同样遵循“先计数后选路”的原则。只有在最终选中 prefilter 路径时，才将 bitset 结果物化为候选 ID 列表。

若标准化后存在越界标签 ID，intersect 直接忽略这些标签；若所有标签都越界，则 $m(F) = 0$ 。该规则与实验实现保持一致，并避免因数据集 label universe 扩展不一致而导致运行时崩溃。

6.5 查询期 scratch 复用

第一阶段每个查询线程必须复用一份 HybridQueryScratch，其内部包含：

1. 长度为 words_per_label 的 `std::vector<uint64_t>` 位图缓冲区；
2. 候选 ID 输出缓冲区 `std::vector<uint32_t>`；
3. 标准化后的标签缓冲区 `std::vector<uint32_t>`。

只有在路由结果为 prefilter 时，才允许填充候选 ID 输出缓冲区；graph-only 路径只保留计数结果，不进行物化。

7 阈值初始化与持久化

7.1 初始阈值生成

第一次索引构建完成后，必须执行一次独立的后处理标定工具，用来生成初始候选规模阈值 τ_m 。该工具不嵌入 `build_disk_index(...)` 内部，原因是现有 build 路径不持有查询集、query labels 和回归参数。第一阶段的正式交付顺序固定为：

1. 执行 build，生成 `_disk.index` 和 `_labels.densebit`；
2. 执行 `tests/calibrate_hybrid_threshold.cpp`，读取正式索引和显式提供的校准查询集；
3. 生成 `_hybrid.meta`；
4. 之后才允许对外提供 hybrid 自动路由能力。

标定工具采用以下输入结构：

```
struct HybridCalibrationInput {
    HybridFilterKind filter_kind;
    std::string query_bin;
    std::string query_label_file;
    uint32_t sample_limit;
};

struct HybridCalibrationConfig {
    uint64_t k;
    uint32_t mem_L;
    uint64_t l_search;
    uint32_t beamwidth;
    std::vector<HybridCalibrationInput> datasets;
};
```

标定算法固定如下：

1. 使用单线程执行，避免与前台检索竞争资源；
2. 对每个 HybridCalibrationInput，使用固定随机种子 20260423 从查询集中均匀采样不超过 sample_limit 条查询；
3. 对每条采样查询先计算 $m(F)$ ，再分别以 kForcePrefilter 和 kForceGraphOnly 执行一次查询；
4. 对 $m(F) = 0$ 的查询只记录诊断信息，不纳入阈值桶统计；
5. 对 $m(F) > 0$ 的查询，按“最小的 2^p 使得 $m(F) \leq 2^p$ ”进行分桶；
6. 每个分桶分别计算 prefilter 与 graph-only 的 p50 延迟；
7. 只有样本数不少于 8 的桶可以参与阈值决策；
8. 初始阈值 τ_m 定义为满足 $p50_prefilter \leq p50_graph$ 的最大桶上界；
9. 若不存在满足条件的桶，则写入 $\tau_m = 0$ ，此时自动路由只对 $m(F) = 0$ 使用 fast-return，其余全部走 graph-only；
10. 将 τ_m 、标定时的 N_{calib} 、标定配置和分桶统计写入 `<index\_prefix>\_hybrid.meta`。

7.2 阈值元数据

第一阶段不扩展 SSDIndexMetadata 的磁盘头，而是定义固定二进制文件 `<index\_prefix>\_hybrid.meta`。该文件采用小端序，布局固定如下：

```
constexpr uint64_t kHybridMetaMagic = 0x4859425249444d54ULL;
constexpr uint32_t kHybridMetaVersion = 1;
```

```
struct HybridMetadataHeaderV1 {
    uint64_t magic;
    uint32_t version;
    uint32_t header_bytes;
    uint64_t flags;
    uint64_t route_selector_mask;
    uint64_t tau_m;
    uint64_t n_calib;
    uint64_t n_live_snapshot;
    uint64_t threshold_version;
    uint64_t calib_epoch_sec;
    uint64_t calib_query_count;
    uint64_t calib_bucket_count;
    uint64_t calib_k;
    uint64_t calib_mem_L;
    uint64_t calib_beamwidth;
```

```
uint64_t calib_l_search;
uint64_t densebit_npoints;
uint64_t densebit_nlabels;
uint64_t densebit_words_per_label;
uint64_t densebit_nnz;
uint64_t reserved[13];
}; // 256 bytes
```

```
struct HybridCalibrationBucketV1 {
uint64_t candidate_upper_bound;
uint64_t query_count;
uint64_t prefilter_p50_us;
uint64_t graph_p50_us;
uint64_t reserved;
}; // 40 bytes
```

文件组织规则固定为：

1. 文件起始 256 字节为 HybridMetadataHeaderV1；
2. 随后紧跟 calib_bucket_count 条 HybridCalibrationBucketV1；
3. 分桶记录按 candidate_upper_bound 升序排列；
4. 文件总长度必须满足

$$\text{file_bytes} = 256 + 40 \times \text{calib_bucket_count}.$$

字段含义固定如下：

字段	类型	含义
magic	uint64_t	固定魔数 kHybridMetaMagic。
version	uint32_t	固定版本号 1。
header_bytes	uint32_t	固定为 256。
flags	uint64_t	状态位。bit0=metadata 有效； bit1=calibration 有效；bit2= 允许 prefilter； bit3= 待重定位；bit4= 后台重定位进行中； 其余保留。
route_selector_mask	uint64_t	bit0 表示 intersect 已标定，bit1 表示 subset 已标定。第一阶段允许值仅为 1、2 或 3。
tau_m	uint64_t	当前候选规模阈值。
n_calib	uint64_t	上次标定时的 live 向量总数。第一阶段固定等 于 meta_.npoints。
n_live_snapshot	uint64_t	metadata 写入时的 live 向量总数。第一阶段 与 n_calib 相同。

threshold_version	uint64_t	阈值版本号。第一次标定写 1，每次成功重定位加 1。
calib_epoch_sec	uint64_t	UNIX 时间戳，单位秒。
calib_query_count	uint64_t	参与标定的总查询数。
calib_bucket_count	uint64_t	分桶记录数量。
calib_k	uint64_t	标定时的 k。
calib_mem_L	uint64_t	标定时的 mem_L。
calib_beamwidth	uint64_t	标定时的 beamwidth。
calib_l_search	uint64_t	标定时的 l_search。
densebit_npoints	uint64_t	用于交叉校验 _labels.densebit 文件头的 npoints。
densebit_nlabels	uint64_t	用于交叉校验 _labels.densebit 文件头的 nlabels。
densebit_words_per_label	uint64_t	用于交叉校验 _labels.densebit 文件头的 words_per_label。
densebit_nnz	uint64_t	用于交叉校验 _labels.densebit 文件头的 nnz。
reserved[13]	uint64_t[13]	预留字段，第一阶段全部写 0。

HybridCalibrationBucketV1 的含义固定如下：

1. candidate_upper_bound: 该桶的候选规模上界；
2. query_count: 落入该桶的查询数；
3. prefilter_p50_us: 该桶中强制 prefilter 的 p50 延迟；
4. graph_p50_us: 该桶中强制 graph-only 的 p50 延迟；
5. reserved: 第一阶段写 0。

metadata 加载必须执行以下验证：

1. magic、version、header_bytes 匹配常量；
2. 文件长度与 calib_bucket_count 一致；
3. flags 的 bit0 和 bit1 均已置位；
4. route_selector_mask 仅使用低两位；
5. densebit_* 字段与 <index_prefix>_labels.densebit 文件头完全一致；
6. n_calib 与当前只读索引的 meta_._npoints 一致，否则禁用第一阶段自动路由。

第一阶段的只读索引中， N_{calib} 和 N_{live} 都取当前索引的 meta_._npoints。只有进入第二阶段后， N_{live} 才改由动态更新路径维护。

7.3 DenseBitset sidecar 文件格式

第一阶段沿用实验实现的 payload 组织方式，但将命名规则固定为 `<index\_prefix>\_labels.densebit`。文件头如下：

```
constexpr uint64_t kDenseBitsetMagic = 0x54494245534E4544ULL;
constexpr uint64_t kDenseBitsetVersion = 1;

struct DenseBitsetFileHeaderV1 {
    uint64_t magic;
    uint64_t version;
    uint64_t npoints;
    uint64_t nlabels;
    uint64_t words_per_label;
    uint64_t nnz;
}; // 48 bytes
```

payload 组织固定为：

1. 紧跟 48 字节头部；
2. 按 label-major 顺序存放 `nlabels * words_per_label` 个 `uint64_t`；
3. 第 `label` 行第 `word_idx` 个 word 的最低位对应 point id `64 * word_idx + 0`；
4. 文件总长度必须满足

$$\text{file_bytes} = 48 + 8 \times \text{nlabels} \times \text{words_per_label}.$$

正式 build 过程中的 sidecar 生成规则固定如下：

1. 输入必须是 `label\_file.spmat`；
2. sidecar builder 只读取 `.spmat` 的 `header`、`indptr`、`indices` 和 `data`；
3. 对每个非零元素 (`point_id`, `label_id`) 置位；
4. 完整写入 `<index\_prefix>\_labels.densebit.tmp` 后原子重命名；
5. 生成完成后允许删除原始 `label\_file.spmat`，因为运行时不再依赖其路径。

8 第二阶段：阈值重定位机制

本节内容不属于第一阶段阻塞项。只有在动态索引补齐标签变更接口、DenseBitset live 状态写回能力以及 N_{live} 统一状态源后，才进入本节实现。

8.1 前置运行时结构

第二阶段开始前，动态路径必须先补齐以下运行时结构：

```
struct HybridForegroundCounters {
std::atomic<uint32_t> active_queries{0};
std::atomic<uint32_t> waiting_queries{0};
std::atomic<uint32_t> active_high_priority_tasks{0};
std::atomic<bool> background_recalibration_disabled{false};
};

struct HybridForegroundBudget {
uint32_t active_queries_low_watermark = 1;
uint32_t waiting_queries_low_watermark = 0;
};

enum class HybridRecalibrationState : uint8_t {
kIdle = 0,
kPending = 1,
kRunning = 2
};
```

这三类结构统一挂在 `DynamicSSDIndex` 上，且 `waiting_queries` 必须在公共查询入口入队前递增、出队后递减，不能由后台线程估算。

8.2 唯一触发条件

阈值重定位的唯一触发条件定义为 `live` 向量总数相对上一次标定时的向量总数发生超过 10% 的变化。触发条件写为

$$\frac{|N_{\text{live}} - N_{\text{calib}}|}{N_{\text{calib}}} > 0.10.$$

除这一条件外，不再引入任何其他自动触发条件。标签变化不触发阈值重定位，维护事件也不单独触发阈值重定位。

8.3 后台重定位执行方式

当触发条件成立后，状态机固定如下：

1. 若当前状态为 `kIdle`，则切换为 `kPending`；若当前状态已经是 `kPending` 或 `kRunning`，则不重复排队；
2. 后台线程仅有一个，由 `DynamicSSDIndex` 在第一次进入 `kPending` 时懒创建；
3. 后台线程从 `kPending` 切换到 `kRunning` 后，按第一阶段相同的标定算法重新生成新的 τ_m ；
4. 每次实际执行一条 `benchmark` 查询前，都必须先做一次前台负载检查；
5. 后台线程在每次单条 `benchmark` 查询时只持有一次查询所需的共享锁，不允许在整个标定期间长期阻塞 `merge`；

6. 标定成功后，原子写回新的 `_hybrid.meta`，同步更新内存中的 `threshold_version`、 τ_m 和 N_{calib} ，再切回 `kIdle`；
7. 标定失败时保留旧阈值，并从 `kRunning` 回到 `kPending`，等待下一次空闲窗口。

后台重定位期间，前台查询继续使用旧阈值工作，待新阈值生成并持久化后再切换。

8.4 前台负载检查

每条后台 benchmark 查询开始前，都必须检查一次前台负载。若前台处于高负载，则本条 benchmark 查询不执行，后台任务保留在 `kPending` 或 `kRunning` 状态并通过 `condition_variable` 等待下一次调度。第二阶段的负载检查条件固定为：

1. `active_queries <= active_queries_low_watermark`;
2. `waiting_queries <= waiting_queries_low_watermark`;
3. `active_high_priority_tasks == 0`;
4. `background_recalibration_disabled == false`。

第一阶段不实现上述计数器，因此 M6 明确是第二阶段工作，不能前移。

9 第二阶段：更新路径配套要求

当前仓库中的动态索引只提供插入、懒删除和合并删除能力，尚未提供标签变更接口，也没有 DenseBitset sidecar 的正式写回路径。因此更新路径相关内容单独列为第二阶段，并增加以下前置要求：

1. 新增标签变更接口：

```
int DynamicSSDIndex<T, TagT>::update_labels(const TagT& tag,
const uint32_t* labels,
uint32_t label_count);
```

2. 新增唯一 live 计数源 `std::atomic<uint64_t> live_point_count_`，并约定所有插入、删除和标签变更完成后都只能更新这一份状态；
3. 新增 in-memory 可变 DenseBitset 运行时，用于查询期实时计数；磁盘上的 `<index\_prefix>\_labels.densebit` 只在 checkpoint 或 merge 后通过原子重写落盘；
4. 在 `<index\_prefix>\_hybrid.meta` 的 `flags` 中启用 `bit3` 和 `bit4`，分别表示待重定位和重定位进行中。

满足上述前置条件后，第二阶段的更新路径需要同时维护路由输入与阈值元数据：

1. 插入操作：对新 ID 的所有标签行置位，并将 `live_point_count_` 加 1；
2. 删除操作：对该 ID 的所有当前标签行清位，并将 `live_point_count_` 减 1；
3. 标签变更操作：对 old-only 标签清位，对 new-only 标签置位，`live_point_count_` 不变化；

4. 标签变化只更新候选规模计算输入，不触发阈值重定位；
5. 每次插入或删除完成后都要检查一次是否满足 10% 的向量数量变化阈值；
6. 若满足条件，则只把状态切到 `kPending`，不阻塞前台查询。

10 验证计划

第一阶段实现完成后，至少执行以下验证：

10.1 构建与集成验证

1. 主工程 `src/CMakeLists.txt` 能成功编译包含 `filter/*.cpp` 的正式库；
2. 正式测试驱动可以直接调用统一 hybrid 查询入口；
3. 主工程中的正式 binary 可以独立完成 sidecar 生成、smoke 与回归，不再依赖 `hybrid_rebuild_context` 中的 binary 或脚本；
4. 删除 `hybrid_rebuild_context` 后，仓库仍能完成构建与核心验证。

10.2 路由正确性验证

1. 单标签查询可基于 `DenseBitset popcount` 正确选路；
2. `subset` 查询可通过 `bitset` 按位与加 `popcount` 正确计算 $m(F)$ ；
3. `intersect` 查询可通过 `bitset` 按位或加 `popcount` 正确计算 $m(F)$ ；
4. 空候选查询能够走 `prefilter-fast-return`，且不会进入 PQ shortlist；
5. 路由决策日志或 `HybridQueryStats` 中能看到 $m(F)$ 、 τ_m 、阈值版本和最终路径选择。

10.3 第一阶段阈值初始化验证

1. 构建后轻量 benchmark 仅针对 `intersect/subset` 查询执行；
2. τ_m 与 N_{calib} 被正确写入 `<index\_prefix>\_hybrid.meta`；
3. metadata 中的 `densebit_*` 字段与 `<index\_prefix>\_labels.densebit` 文件头一致；
4. 重新加载索引后可正确读取 companion metadata，并驱动统一路由。

10.4 文件格式验证

1. `<index\_prefix>\_labels.densebit` 的文件长度满足头部定义；
2. `<index\_prefix>\_hybrid.meta` 的文件长度满足 $256 + 40 * \text{calib_bucket_count}$ ；
3. metadata 损坏、sidecar 损坏或两者不一致时，`SSDIndex::load()` 会禁用 hybrid runtime 并回退到 `graph-only`；
4. build 完成后删除原始 `label\_file.spmat`，正式 hybrid binary 仍可完成加载和查询。

10.5 第二阶段阈值重定位验证

1. 当 N_{live} 与 N_{calib} 的差值超过 10% 时, 后台重定位任务被正确触发;
2. 后台重定位始终以单线程执行;
3. 每条后台 benchmark 查询前都执行前台负载检查;
4. 当负载检查失败时, 后台任务能暂停并在后续继续;
5. 阈值更新后, τ_m 、 N_{calib} 、threshold_version 和 flags 被正确持久化。

11 模块设计与实施接口

本节按模块组织, 给出实施时必须遵守的目标文件、核心职责、主接口和验收条件。后续实现只允许在这些边界内细化, 不允许重新定义文件命名、磁盘格式或阈值协议。

11.1 模块 M1: DenseBitset 运行时模块

状态: 未开始。

目标文件: include/filter/densebit_index.h、src/filter/densebit_index.cpp。

核心职责:

1. 加载并校验 <index_prefix>_labels.densebit;
2. 对 intersect/subset 实现统一的计数与物化;
3. 提供线程内 scratch 复用接口;
4. 保证候选物化结果按 point id 升序输出。

主接口:

```
class DenseBitsetIndex {
public:
static DenseBitsetIndex load(const std::string& sidecar_path);
uint64_t count_candidates(HybridFilterKind kind,
Span<const uint32_t> labels,
HybridQueryScratch* scratch) const;
void materialize_candidates(HybridFilterKind kind,
Span<const uint32_t> labels,
HybridQueryScratch* scratch,
std::vector<uint32_t>* output) const;
};
```

验收条件: 通过路由正确性验证和文件格式验证。

11.2 模块 M2: Prefilter 执行器

状态: 未开始。

目标文件: include/filter/hybrid_route.h、src/search/hybrid_prefilter.cpp。

核心职责:

1. 接收已经物化的候选 ID 列表;
2. 复用实验实现中的 PQ 打分和 SSD 精排逻辑;
3. 对空候选集直接返回;
4. 固定使用如下 rerank_L 公式:

$$L_r(m) = \min(\max(T(s), k), m), \quad s = m/N,$$

其中

$$T(s) = \begin{cases} 96, & s \leq 0.005 \\ 128, & 0.005 < s \leq 0.02 \\ 160, & 0.02 < s \leq 0.1 \\ 192, & s > 0.1 \end{cases}$$

且 N 在第一阶段取 meta_ .npoints。

主接口:

```
size_t run_hybrid_prefilter(...,
const std::vector<uint32_t>& candidate_ids,
uint64_t total_points,
QueryStats* stats);
```

验收条件: 强制 prefilter 路径在正式驱动中可独立跑通, 并与实验实现的 recall/延迟量级一致。

11.3 模块 M3: 统一 Hybrid 查询入口与路由器

状态: 未开始。

目标文件: include/ssd_index.h、include/filter/hybrid_route.h、src/search/hybrid_search.cpp。

核心职责:

1. 实现 SSDIndex::hybrid_search(...);
2. 自动加载 HybridRuntime;
3. 提供 force_route 覆盖, 用于测试和标定;
4. 对不可用 runtime、损坏 metadata 或不支持的过滤类型执行稳定回退。

验收条件: 自动路由、强制 prefilter、强制 graph-only 三条路径都能在同一驱动中复用同一索引前缀完成查询。

11.4 模块 M4: Metadata 模块

状态: 未开始。

目标文件: include/filter/hybrid_metadata.h、src/filter/hybrid_metadata.cpp。

核心职责:

1. 按 7.2 节的固定格式读写 <index_prefix>_hybrid.meta;
2. 完成 header、bucket records 和 sidecar 头的交叉校验;
3. 提供 write_atomically();
4. 提供 disable_routing(), 在 metadata 无效时显式关闭 hybrid 路由。

主接口:

```
class HybridMetadata {
public:
static HybridMetadata load(const std::string& meta_path);
void validate_against_densebit(const DenseBitsetFileHeaderV1&) const;
void write_atomically(const std::string& meta_path) const;
};
```

验收条件: metadata 文件能被独立测试程序完整 round-trip, 且损坏场景下会触发禁用回退。

11.5 模块 M5: 初始阈值标定与 build 接线

状态: 未开始。

目标文件:src/utils/index_build_utils.cpp、tests/build_disk_index.cpp、tests/build_pipnn_index.cpp、src/pipnn.cpp、tests/calibrate_hybrid_threshold.cpp。

核心职责:

1. 将 <index_prefix>_labels.densebit 生成逻辑接入正式 build;
2. 保证 build 阶段在缺失 label_source_file 时直接失败;
3. 新增后处理标定工具, 按 7.1 节算法生成 _hybrid.meta;
4. 保证 build 后没有 metadata 时, hybrid 自动路由默认关闭。

验收条件: 正式 build + 正式标定工具在不依赖实验目录脚本的情况下可生成完整 hybrid 产物集合。

11.6 模块 M6: 后台阈值重定位与前台负载检查

状态: 第二阶段未开始。

目标文件: include/dynamic_index.h、src/update/dynamic_index.cpp、相关后台调度实现文件。

核心职责:

1. 接入 HybridForegroundCounters 和 HybridRecalibrationState;

2. 实现单线程后台标定；
3. 每条后台 benchmark 查询前执行固定负载检查；
4. 仅通过 bit3/bit4 和 threshold_version 更新 metadata 状态。

验收条件：重定位触发、暂停、恢复、成功写回和失败保留旧值都可通过专项测试验证。

11.7 模块 M7：更新路径集成

状态：第二阶段未开始。

目标文件：include/dynamic_index.h、src/update/dynamic_index.cpp、label 子系统相关文件。

核心职责：

1. 新增 update_labels(...) 接口；
2. 维护唯一的 live_point_count_；
3. 维护 in-memory 可变 DenseBitset；
4. 在 checkpoint 或 merge 后原子重写 <index_prefix>_labels.densebit。

验收条件：插入、删除、标签变更后，候选计数与查询结果始终与 live 状态一致。

11.8 模块 M8：正式测试驱动与实验目录删除

状态：未开始。

目标文件：tests/search_disk_index_hybrid.cpp、tests/CMakeLists.txt、src/CMakeLists.txt。

核心职责：

1. 新增正式 hybrid 查询驱动；
2. 覆盖 auto、force-prefilter、force-graph 三种模式；
3. 在构建系统中移除实验目录依赖；
4. 满足实验目录删除条件后，删除 hybrid_rebuild_context。

验收条件：删除实验目录后，正式 build、标定、查询和回归均仅依赖主源码树。

12 实施顺序

正式实施顺序固定如下：

第一阶段：

1. 先改 build 路径：接入 <index_prefix>_labels.densebit 的正式生成逻辑；
2. 实现 M1 DenseBitset 运行时与 M4 metadata 模块，打通 load 校验链路；
3. 实现 M2 prefilter 执行器与 M3 hybrid_search(...)；

4. 新增 `tests/search_disk_index_hybrid.cpp`, 完成自动路由和强制路径回归;
5. 新增 `tests/calibrate_hybrid_threshold.cpp`, 完成 build 后阈值标定;
6. 在正式 binary、正式测试、sidecar 生成和阈值流程稳定后, 整体删除 `hybrid_rebuild_context` 目录及其所有残余引用。

第二阶段:

1. 先补齐 `update_labels(...)`、`live_point_count_` 和前台负载计数器;
2. 再接入 in-memory 可变 `DenseBitset` 与 `checkpoint/merge` 写回;
3. 最后实现后台单线程重定位与前台负载检查。

13 交付结果

本计划完成后, 工程侧应交付以下内容:

第一阶段交付:

1. 正式源码树中的 hybrid 查询实现, 仅覆盖 `intersect/subset`;
2. 正式构建系统中的 hybrid 编译接线;
3. 正式测试驱动与回归验证;
4. 固定命名的 `<index\_prefix>\_labels.densebit` 和 `<index\_prefix>\_hybrid.meta` 文件格式;
5. 基于 companion metadata 的 build 后初始阈值标定流程;
6. 仓库中不再保留 `hybrid_rebuild_context` 目录。

第二阶段交付:

1. 动态索引下的 N_{live} 维护与 `DenseBitset` live 状态更新;
2. 仅由向量数量变化 10% 触发的后台阈值重定位流程。